

File Guide — Egyptian Currency Detection

A walk-through of every file and folder in the project, what it does, and when it matters.

Top-level files

README.md

The user-facing entry point. Covers install, dataset preparation (including the bootstrap step), training, evaluation, detection (image / folder / video / webcam), the JSON output schema, and tuning tips. **Read this first** if you're new to the repo.

PROJECT_REPORT.md

The deliverable graduation-style report. Documents the project goal, the classifier-vs-detector decision, the YOLOv8 training pipeline, run history (including the failures and recoveries), final mAP / precision / recall numbers, end-to-end pipeline test, deliverables list, and limitations. **This is the document to hand in / show.**

PROJECT_REPORT.pdf

The same report rendered to PDF (via `src/report_to_pdf.py`). Has styled headings, page numbers, and tables. Open this for sharing or printing.

Progress_Report.pdf

Older progress report from the original classification phase of the project. Kept for historical reference. Not regenerated by anything in this repo.

requirements.txt

The full Python dependency list:

- `ultralytics` — YOLOv8 detector + training engine
- `torch`, `torchvision` — deep-learning backend
- `opencv-python` — image / video / webcam I/O
- `numpy`, `Pillow`, `matplotlib`, `PyYAML` — supporting libraries

Install with `pip install -r requirements.txt`.

.gitignore

Tells git which paths to ignore (e.g., `.venv`, large outputs). Keeps the repository clean.

yolov8n.pt

The pretrained YOLOv8-nano base weights, auto-downloaded by ultralytics the first time you train. Reused by subsequent trainings. Safe to delete — it will re-download automatically.

.venv/

The local Python virtual environment with all installed packages. Ignored by git. Don't commit this.

src/ — All current project code

This is where every script you actually run lives. All scripts use `config.py` as their single source of truth for paths and hyperparameters.

src/config.py

Centralized configuration. One file changes paths, class names, denominations, training hyperparameters, and detection thresholds for the whole project. Other scripts import from here. Key things it defines:

- `CLASS_NAMES` — the 7 classes (1, 5, 10, 20, 50, 100, 200 EGP).
- `DENOMINATIONS` — the EGP value mapping used to compute `total`.
- `DATA_YAML` — path to the YOLO dataset spec.
- `DEFAULT_WEIGHTS` — points at `outputs/runs/train2/weights/best.pt`.
- `IMG_SIZE`, `BATCH_SIZE`, `EPOCHS`, `CONF_THRESHOLD`, `IOU_THRESHOLD`.

src/utils.py

Shared helpers used by `detect.py` (and reusable elsewhere):

- `load_model(weights_path)` — lazy-imports ultralytics, loads a YOLO model.
- `parse_yolo_result(...)` — converts ultralytics' result objects into the

project's detection-dict format.

- `summarize(detections)` — counts notes per class and totals their EGP value.
- `draw_detections(...)` — overlays bounding boxes + class labels on a frame.
- `draw_summary_panel(...)` — top-left translucent panel with per-class

counts and the running total.

- `build_output_dict(...)` / `save_json(...)` — produces and saves the

final `{detections, counts, total}` JSON.

src/bootstrap_yolo_dataset.py

Converts the original folder-per-class dataset into YOLO format. Reads images from `<src>/{train,valid,test}/<class>/*.jpg` and writes:

- the images to `data_yolo/images/{train,val,test}/`
- a YOLO `.txt` per image to `data_yolo/labels/{train,val,test}/` with a

full-image pseudo-box (`<class_id> 0.5 0.5 1.0 1.0`) Also collapses the 9 source folders (e.g., 10 and 10 (new)) into the project's 7 unified classes. Run this once on a fresh source dataset.

src/train.py

Trains the YOLOv8 detector. Wraps `ultralytics.YOLO.train()` with sensible defaults for currency-photo augmentation (HSV jitter, rotation, perspective, scale, mixup, mosaic) and exposes CLI flags for epochs, batch, imgsiz, device, and `--low-mem` (disables mosaic + mixup to fit small CPUs). Saves training curves and weights to `outputs/runs/<name>/`.

src/detect.py

The inference entry point. One script, four input modes:

- `--image <path>` — single image
- `--image-dir <path>` — every image in a folder
- `--video <path>` — a video file → annotated mp4
- `--webcam` — live camera feed

Each mode produces an annotated image / video and a JSON file in `outputs/detections/`, matching the project's required schema:

```
{
  "detections": [{ "class": ..., "confidence": ..., "bbox": [...] }, ...],
  "counts":      { "100_EGP": 2, ... },
  "total":       250
}
```

Tunable via `--conf` (min confidence) and `--iou` (NMS threshold).

`src/evaluate.py`

Computes detection metrics on a dataset split. Calls `ultralytics.YOLO.val()` on `data_yolo/images/test` and reports mAP@0.5, mAP@0.5:0.95, precision, recall, and per-class breakdowns. Saves PR / F1 / confusion-matrix plots to `outputs/runs/eval/`.

`src/export_ptl.py`

Exports the trained model to TorchScript Lite (.ptl) for mobile. Steps: `best.pt` → ultralytics export to TorchScript → `optimize_for_mobile` → `_save_for_lite_interpreter`. Produces `outputs/runs/train2/weights/best.ptl` (~11.5 MB) for use in Android / iOS apps.

`src/report_to_pdf.py`

Renders `PROJECT_REPORT.md` into `PROJECT_REPORT.pdf`. Custom markdown parser plus reportlab Platypus flowables: titled cover, blue section headers, striped tables, code blocks, page numbers. Re-run any time the markdown report changes.

`data_yolo/` — The YOLO-format dataset

`data_yolo/data.yaml`

The dataset spec. Tells ultralytics where to find images, labels, and the 7 class IDs. Uses an absolute `path:` because ultralytics resolves relative paths against its global datasets directory, not against the yaml file. **Don't make path: relative again** — that broke the first training run.

`data_yolo/images/{train,val,test}/`

The actual JPGs the model sees. 2637 / 760 / 290 images. Filenames are prefixed with the source folder slug (e.g., `100__100.1.jpg`, `10_new__img_3.jpg`) so old/new variants don't collide.

`data_yolo/labels/{train,val,test}/`

One `.txt` per image with YOLO-format boxes: `<class_id> cx cy w h` (all normalized to [0,1]). All current labels are full-image pseudo-boxes — the next improvement step is replacing some of these with hand-drawn real boxes for multi-note photos.

outputs/ — Everything generated by the project

outputs/best_model.pth

The **legacy classifier** checkpoint — `CurrencyMobileNet` weights from the original (pre-YOLO) project. Used only by scripts in `legacy/`. Not loaded by any new code. Safe to delete if you commit fully to YOLO.

outputs/runs/train2/

The main trained model + everything from training. Subfolders / files of interest:

- `weights/best.pt` — the **best-mAP checkpoint**, the one `detect.py`

loads by default.

- `weights/last.pt` — last-epoch checkpoint. Useful for resuming.
- `weights/best.torchscript` — intermediate TorchScript export

(produced by `export_ptl.py`).

- `weights/best.ptl` — **mobile model** (TorchScript Lite, 11.5 MB).
- `results.csv` — per-epoch loss + mAP/P/R numbers (used by the report).
- `results.png` — training curves plot (loss + mAP over epochs).
- `BoxPR_curve.png`, `BoxF1_curve.png`, `BoxP_curve.png`, `BoxR_curve.png` — precision/recall/F1 curves.
- `confusion_matrix.png`, `confusion_matrix_normalized.png` — per-class

confusion matrices.

- `labels.jpg` — visualization of label distribution and box positions

across the dataset.

- `train_batch*.jpg` — sample augmented training batches.
- `val_batch*_labels.jpg` / `val_batch*_pred.jpg` — side-by-side

ground-truth vs. predictions on val batches.

- `args.yaml` — the exact arguments this run was started with (handy for reproducing).

outputs/runs/eval/

Output from `src/evaluate.py`: PR plots and confusion matrix evaluated on the test split (290 images, mAP50 = 0.995).

outputs/detections/

Where every `detect.py` run drops its annotated images and JSON files. Filenames use `<image_stem>_annotated.jpg` and `<image_stem>.json`. For folder runs, results land in a same-named subfolder.

legacy/ — The original classification project (preserved as-is)

These files are the project as it existed *before* the YOLO upgrade. They are kept so you can still run the original classifier if you ever need to. None of them is imported by anything in `src/`.

`legacy/model.py`

`CurrencyMobileNet` — a from-scratch MobileNetV2-style classifier with ~2.2M parameters. Outputs a single class per image. Was originally trained on `legacy/data/`.

`legacy/dataset.py`

Folder-based image classification dataset: loads `data/<split>/<class>/*.jpg`, applies torchvision augmentation (rotation, color jitter, perspective, random erasing).

`legacy/train.py`

Original training loop with mixed-precision (AMP), weighted sampling, label smoothing, cosine LR, early stopping.

`legacy/evaluate.py`

Evaluates the classifier on a split: accuracy, precision, recall, F1, mAP-style score, confusion matrix.

`legacy/infer.py`

Single-image classification CLI. Outputs top-1 / top-K predictions. **Could not count multiple notes** — that's why the project moved to YOLO.

`legacy/camera.py`

A pre-YOLO attempt at multi-note detection: OpenCV adaptive-threshold + contour analysis to find rectangular regions, then classify each crop. Worked on simple backgrounds but was fragile under clutter, overlap, or varied lighting. Made obsolete by `src/detect.py`.

`legacy/config.py`

Original hyperparameter / path file for the classifier (9 classes: 1, 5, 10, 10 (new), 20, 20 (new), 50, 100, 200).

`legacy/utils.py`

`MetricTracker`, `EarlyStopping`, `TrainingLogger`, checkpoint save/load, metric printing, prediction-grid visualization. Specific to the classifier; not used by YOLO scripts.

`legacy/export_ptl.py`

Old TorchScript Lite exporter for `CurrencyMobileNet`. Replaced by the YOLO-aware `src/export_ptl.py` for the current model.

`legacy/generate_report.py`

Generated a per-class report for the classifier. Not used by the new pipeline.

File-importance cheat sheet

If you only have time to read a handful of files, this is the order:

- `README.md` — how to run anything.

- `src/config.py` — what the project's defaults are.
- `src/detect.py` — the headline feature in 200 lines.
- `src/utls.py` — count + total + JSON logic.
- `src/train.py` — how the model was trained.
- `PROJECT_REPORT.md` — the story, the numbers, the limitations.

Files you can ignore unless you're investigating a specific problem:

- Anything in `legacy/` (only matters if you're reviving the classifier).
- The `*.png` plots in `outputs/runs/train2/` are useful screenshots for reports, not for development.
- `Progress_Report.pdf` is historical.